2. Funciones

Sitio:	Centros Granada 26	Imprimido por:	Triay Orfila, Marc
Curso:	Desarrollo web en entorno servidor	Día:	lunes, 5 de octubre de 2026, 17:13
Libro:	2. Funciones		

Tabla de contenidos

1. Introducción

2. Funciones predefinidas

3. Funciones definidas por el usuario

4. Ámbito de las variables

5. Argumentos predeterminados o por defecto

6. Argumentos con nombre

7. Lista de argumentos variable

8. Argumentos con tipo definido

9. Funciones con tipo definido

10. Paso de argumentos por valor

11. Paso de argumentos por referencia

12. Funciones como argumentos

13. Funciones anónimas

14. Funciones de flecha

15. Funciones variables

1. Introducción

Una función es un conjunto de instrucciones que realiza una tarea concreta.

Las funciones pueden recibir argumentos, es decir, los datos que necesitan para llevar a cabo su contenido.

Las funciones pueden devolver o no un valor. En PHP no es necesario declarar el tipo de dato que devuelve una función, y, de hecho, puede devolver tipos de datos diferentes según el caso. Por ejemplo, muchas funciones devuelven FALSE en caso de error y no devuelven un *boolean* si no hay error.

Al desarrollar aplicaciones complejas, a menudo, hay instrucciones que se repiten. Para mejorar el código las funciones son una solución para poder reutilizar el código.

Cada función en PHP se considera como una expresión, y por tanto, se puede almacenar su resultado en una variable o formar arte de una expresión más compleja.

2. Funciones predefinidas

Las más utilizadas están relacionadas con las variables.

Como en PHP las variables no se declaran explícitamente hay una serie de funciones que permiten conocer el estado y el tipo de datos de una variable.

Entre las más importantes tenemos:

- is_null($var)
- isset($var)
- unset($var)
- empty($var)
- is_int($var)
- print_r($var)
- var_dump($var)

Funciones para obtener la fecha:

- date()
- strftime()
- setlocate()

Algunas funciones de cadenas:

- strlen($cad)
- strcmp($cad1,$cad2)
- strtolower($cad)
- strtoupper($cad)
- str($cad1,$cad2)
- strpos($cad)

Las siguientes se refieren a cadenas pero sólo están disponibles en PHP 8:

- str_contains($cad)
- str_starts_with($cad)
- str_ends_with($cad)

Estos son sólo algunos ejemplos , puedes consultar la guía online de PHP para saber con detalle de todas las funciones.

3. Funciones definidas por el usuario

Su **sintaxis:**

```
function nombre(argumentos){
   <instrucciones>
}
```

La función puede tener o no argumentos que se declaran en la cabecera separados por comas.

Haremos uso de la palabra reservada **return** para devolver el valor que estamos solicitando cuando se invoca a la función.

Ejemplo:

```php
<?php
function factorial ($numero) {
    $resultado = 1;
    for ($x = $numero; $x > 0; $x--){
        $resultado= $resultado * $x;
    }
    return $resultado;
}

echo "El factorial de 4 es ". factorial(4);
```

4. Ámbito de las variables

El ámbito de una variable es la parte del código en que esta visible. Una variable declarada en un fichero PHP está disponible en ese fichero y en los ficheros que se incluyan desde este.

Por otro lado, las funciones definen un ámbito local, de manera que las variables que se declaran en las mismas solo son accesibles desde la propia función. Esto nos permite tener variables con el mismo nombre en diferentes funciones, haciendo el código reutilizable.

Además, desde la función no se puede acceder a otras variables que no sean las locales o sus argumentos.

Para definir variables globales hay dos opciones. La palabra reservada **global** y la variable predefinida **$GLOBALS**. Las variables globales son accesibles desde cualquier función o fichero de la aplicación.

Veamos un ejemplo en el que hay la variable $a declarada fuera de la función. Y dentro de prueba volvemos a declarar otra variable llamada también $a. Esta última variable es local a la función y se considera distinta.

```
$a = 1;
function prueba(){
$b = $a; /*Dentro de la función no se tiene acceso a la variable $a
anterior

* Por tanto, la variable $a usada en la asignación anterior es una
variable nueva que no tiene valor asignado (su valor es null)
*/
}
```

Si la variable aparece por primera vez dentro de una función, se dice que esa variable es local a la función. Si aparece una asignación fuera de la función, se le considerará una variable distinta.

Si en la función anterior quisieras utilizar la variable $a externa, podrías hacerlo utilizando la palabra reservada *global*. De esta forma le dices a PHP que no cree una nueva variable local, sino que utilice la ya existente.

```php
$a = 1;
function prueba(){

    global $a;
    $b = $a; // En este caso se le asigna a $b el valor 1
}
```

Puedes saber más sobre los ámbitos de las variables consulta la documentación de PHP.
http://www.php.net/manual/es/language.variables.scope.php

5. Argumentos predeterminados o por defecto

Se sustituyen las variables por expresiones de asignación, de forma que , al llamar a la función, podemos hacerlo con varios parámetros. Si alguno es obviado, la variable tendrá como valor el valor por defecto de la definición.

```php
<?php
function capitales($pais, $capital = "Madrid", $habitantes = "muchos")
{
    return "La capital de $pais es $capital y tiene $habitantes
habitantes.\n";
}

echo capitales("España"); // El parámetro país es obligatorio. Los otros
tienen valor por defecto
echo capitales("Portugal", "Lisboa");
echo capitales("Francia", "Paris", "muchisimos");
?>
```

La limitación de este método es que los argumentos tienen un orden, y no podemos saltarnos ninguno.

6. Argumentos con nombre

Nos permiten no tener que seguir un orden concreto a la hora de pasar parámetros a una función, simplemente le pasábamos el nombre del parámetro y su valor . Están disponibles a partir de PHP 8 .

Ya no es necesario que coincidan en el mismo orden los parámetros en la función y en la llamada a la misma.

Ejemplo de función en PHP:

```php
<?php
function moduloDaw($nombre, $profesor, $ies){
    return "El módulo $nombre es impartido por $profesor en el IES $ies";
 }

echo moduloDaw(
    ies: "Francisco Ayala",
    nombre: 'DWES',
    profesor: 'María'
);
```

Por supuesto, podemos seguir realizando la llamada de la forma típica:

```php
echo moduloDaw('DWES', 'María', "Francisco Ayala");
```

7. Lista de argumentos variable

A veces es necesario que una función tenga que gestionar un número variable de argumentos. En este caso, lo indicamos poniendo puntos suspensivos delante del nombre de la variable. En realidad, lo que hace es que actúe como un array donde se capturan todos los argumentos.

La única restricción es que los argumentos variables se definen siempre como último argumento. Los fijos irán al principio y los variables al final.

```php
<?php
function concatena($titulo, …$palabras){
    $resultado = "$titulo <br>";
    foreach($palabras as $palabra){
        $resultado .= $palabra." ";
    }
    return $resultado;
}
echo concatena("Ciclo Formativo:", "Desarrollo", "de", "Aplicaciones",
"Web <br><br>");
echo concatena("Módulo:", "DWES");
```

8. Argumentos con tipo definido

Para que las funciones sean más robustas podemos restringir el tipo de los argumentos. Basta con escribir el tipo antes del nombre del argumento.

```php
<?php
function factorial (int $numero) {
    $resultado = 1;
    for ($x = $numero; $x > 0; $x--){
        $resultado= $resultado * $x;
    }
    return $resultado;
}
```

Si queremos que los tipos de los argumentos sean analizados de forma estricta es necesario indicarlo en la primera línea de cada archivo `.php` para que el propio interprete PHP compruebe los tipos y lance errores si los tipos son incorrectos, mediante la sentencia:

```php
declare(strict_types=1);
```

De esta forma si se pasa un tipo incorrecto, se lanzará una excepción TypeError.

En PHP 8 es posible elegir entre varios tipos de datos dentro de los argumentos de una función:

```php
<?php
funcion sumaNumeros(int|float $sumando1, int|float $sumando2){
    return $sumando1 + $sumando2;
}
```

9. Funciones con tipo definido

Podemos restringir el retorno de una función indicando el tipo después del paréntesis de cierre de los argumentos, precedido de dos puntos.

```php
function tipoDeMedalla(int $posicion): ?string
{
    switch ($posicion) {
        case 1:
            return 'Oro';
        case 2:
            return 'Plata';
        case 3:
            return 'Bronce';
        default:
            return null;
    }
}

echo tipoDeMedalla(2);// Plata

echo tipoDeMedalla(34);// null
```

En el ejemplo anterior, con **el carácter "?"** delante del tipo indicamos la posibilidad de **devolver un tipo concreto o null.**

10. Paso de argumentos por valor

En PHP los argumentos, por defecto, se pasan por valor (o copia). Esto quiere decir que, cuando se llama a una función con una variable como argumento, se crea una variable local a la función en la que se copia el valor del argumento. Si la función modifica el argumento, estos cambios no tienen efecto en la variable original.

```php
<?php
function duplicarPositivo(float $numero): float|string
{
    if ($numero > 0) {
        $resultado = $numero * 2;
        $numero = $resultado;
        return $resultado;
    } else {
        return 'No puedes usar número en negativo o cero';
    }
}
echo duplicarPositivo(-45)."<br>";// 'No puedes usar número en negativo o
cero'
$numero = 12.1;
echo "El doble de ". $numero. " es " .duplicarPositivo($numero);
```

11. Paso de argumentos por referencia

Los argumentos que se pasen por referencia pueden ser modificados durante la ejecución de una función, ya que reciben la variable no una copia.

```php
<?php
function duplicarPositivo(float &$numero): float|string
{
    if ($numero > 0) {
        $resultado = $numero * 2;
        $numero = $resultado;
        return $resultado;
    } else {
        return 'No puedes usar número en negativo o cero';
    }
}

$numero = 12.1;
echo "El doble de ". $numero. " es " .duplicarPositivo($numero)."<br>";
echo "El doble de ". $numero. " es " .duplicarPositivo($numero);
```

Observa cómo cambia el valor de la variable $numero

12. Funciones como argumentos

En PHP podemos **pasar una función como argumento a otra función**.

Cuando una función se pasa a otra función para que esta pueda ejecutarla, hablamos de un callback.

La idea fundamental es:

> **Un callback es una función que se entrega a otra función para que esta pueda ejecutarla cuando la necesite.**

Por ejemplo:

```
function saludar($nombre) {
    return "Hola, $nombre";
}
function ejecutarCallback($callback) {
    return $callback("Ana");
}
echo ejecutarCallback("saludar");
```

Los callbacks permiten que una función sea **más flexible y reutilizable**.

En lugar de programar una función para que siempre haga exactamente lo mismo, podemos indicarle mediante un callback **qué comportamiento queremos que utilice**.

Por ejemplo:

```
function operar($a, $b, $operacion) {
    return $operacion($a, $b);
}
```

Podemos crear diferentes operaciones:

```
function sumar($a, $b) {
    return $a + $b;
}
```

```
function multiplicar($a, $b) {
    return $a * $b;
}
```

Y decidir cuál utilizar:

```
echo operar(5, 3, "sumar");

echo operar(5, 3, "multiplicar");
```

PHP incluye muchas **funciones que utilizan callbacks.**

Una de las más importantes es: `array_map()`

Su función es aplicar una operación a todos los elementos de un array.

Por ejemplo:

```
$nombres = ["Ana", "Pedro", "Laura"];

$resultado = array_map("strtoupper", $nombres);

print_r($resultado);
```

Resultado:

```
Array
(
    [0] => ANA
    [1] => PEDRO
    [2] => LAURA
)
```

En este ejemplo `"strtoupper"` es el callback.

`array_map()` se encarga de ejecutar:

```
strtoupper("Ana");
strtoupper("Pedro");
strtoupper("Laura");
```

En PHP, las funciones callback también pueden ser métodos de un objeto, incluidos los métodos de clase estática.

Los callbacks son muy utilizados en bibliotecas y frameworks porque permiten que una parte del programa pueda **delegar determinadas acciones en el código del programador**.

Por ejemplo, una biblioteca puede proporcionar una función que hace todo el trabajo de procesar datos, pero permitirnos indicar mediante un callback qué queremos hacer con cada resultado.

Suelen emplearse para implementar plugins o módulos de manera limpia y para garantizar su funcionalidad.

13. Funciones anónimas

Las funciones anónimas, o closures, permiten crear funciones que no tienen un nombre específico. De esta forma, podremos declarar una función sin nombre y almacenarla en una variable.

La sintaxis para definir un 'closure' es :

function(){

//implementación

}

Se utilizan, por ejemplo, cuando la función que definimos tiene un uso extremadamente puntual y no va a ser reutilizada. Por eso, la definimos en el mismo lugar donde se usa y de esta forma el código quedará más claro.

```php
<?php
$multiplicador_por_tres = function(int $numero){
    return $numero * 3;
};
echo $multiplicador_por_tres(10);
```

Es muy común usar las funciones anónimas en combinación con los callbacks.

En este ejemplo usaremos una función anónima como callback para multiplicar los elementos de la misma posición de cada uno de los arrays.

```php
$multiplicador = function($a,$b) {
    return $a * $b;
};
$numeros = range (1,10);
$numeros2 = range (1,10);
$lista = array_map($multiplicador, $numeros, $numeros2);
echo implode(" ", $lista);
```

En este ejemplo hemos llamado a la función anónima desde otra función como argumento.

14. Funciones de flecha

Las funciones de flecha (short closures) son una forma más corta de escribir las funciones anónimas que ayuda a que el código sea más legible.

La palabra clave `function` se reemplaza con `fn`, y la palabra clave `return` se omite por completo al definir funciones flecha. Estas solamente contienen una expresión, y esa expresión es el valor de retorno de la función flecha.

Otra característica más de las funciones flecha es que las variables definidas en el ámbito principal están disponibles de manera implícita para ellas sin tener que agregar la palabra clave `use`.

Su forma básica es : **fn (parámetros) => exp**

Veamos un ejemplo en el que tenemos el mismo código representado mediante una función anónima y una función de flecha.

```php
<?php

//Ejemplo Funciones de flecha

$y = 1;
// creamos una función anónima que sirve para sumar
$result1 = function ($x) use ($y) {
    return $x + $y;
};

// la misma función pero expresada mediante función de flecha
// Se capturan las variables por valor automáticamente
// no necesitamos use

$result2 = fn($x) => $x + $y;

echo 'USANDO FUNCIÓN ANÓNIMA <br/>';
echo $y.'+ 3= '. $result1(3).'<br/>';

echo 'USANDO FUNCIÓN DE FLECHA <br/>';
echo $y.'+ 3= '. $result2(3).'<br/>';

?>
```

Debemos de tener en cuenta que:

- No se pueden usar las llaves {}.

- La expresión no lleva ‘;’ al final.
- No se usa return.
- No necesitas el «use» para poder acceder a variables externas (como sí sucede en las funciones anónimas).

Nota: PHP también tiene la palabra clave especial `use`, que te permite acceder a variables del ámbito principal dentro de una función anónima.

15. Funciones variables

En PHP podemos utilizar variables para almacenar el nombre de una función. Basta con añadir en la llamada los paréntesis.

```php
<?php
function saludo_mañana(){
    return "Buenos dias";
}
function saludo_tarde(){
    return "Buenas tardes";
}
function saludo_noche(){
    return "Buenas noches";
}
$horario = "tarde";
$funcion_variable = "saludo_". $horario;
echo $funcion_variable();
```

Nota: Este documento es una recopilación de datos del libro Desarrollo web en entorno servidor de Xabier Ganzábal, el manual de PHP y el libro Curso de PHP 8 y MySQL 8 de Luis Miguel Cabezas.

Reiniciar tour para usuario en esta página